The `StatisticsImplementor` interface defines the contract between the Hibernate internal API and the various custom statistics gathering implementations. For simplicity sake, the following `StatisticsImplementor` interface implementation extends the default `ConcurrentStatisticsImpl` class, as it only needs to override the `connect` and `endTransaction` methods.

```
public class TransactionStatistics extends ConcurrentStatisticsImpl {

    private static final ThreadLocal<AtomicLong> startNanos =
            ThreadLocal.withInitial(AtomicLong::new);

    private static final ThreadLocal<AtomicLong> connectionCounter =
            ThreadLocal.withInitial(AtomicLong::new);

    private StatisticsReport report = new StatisticsReport();

    @Override
    public void connect() {
        connectionCounter.get().incrementAndGet();
        startNanos.get().compareAndSet(0, System.nanoTime());
        super.connect();
    }

    @Override
    public void endTransaction(boolean success) {
        try {
            report.transactionTime(System.nanoTime() - startNanos.get().get());
            report.connectionsCount(connectionCounter.get().get());
            report.generate();
        } finally {
            startNanos.remove();
            connectionCounter.remove();
        }
        super.endTransaction(success);
    }
}
```

The `StatisticsImplementor` is a singleton instance, therefore the `ThreadLocal` counters must be reset after a transaction is ended. When a transaction ends, the report is generated, and both the physical transaction time and the number of connection requests, issued during a particular transaction, are flushed to the log.

Because a Persistence Context can run multiple concurrent transactions, the report is generated at the end of each individual transaction.

To use a custom `StatisticsImplementor` instance, Hibernate requires a `StatisticsFactory` supplied as a configuration property. Taking a `SessionFactoryImplementor` parameter, the `StatisticsImplementor` building process has access to the Hibernate configuration data as well.

```
public class TransactionStatisticsFactory implements StatisticsFactory {
    @Override public StatisticsImplementor buildStatistics(
        SessionFactoryImplementor sessionFactory) {
        return new TransactionStatistics();
    }
}
```

The `hibernate.stats.factory` configuration property must contain the fully qualified name of the `StatisticsFactory` implementation class:

```
<property name="hibernate.stats.factory" value="com.vladmihalcea.book.hpjp.hibernate\
.statistics.TransactionStatisticsFactory" />
```

When running the previous JTA connection release mode example along with this custom statistics collector, the following output is being displayed:

```
type=HISTOGRAM, name=connectionCounterHistogram, count=107,
min=1, max=10000, mean=162.41, stddev=1096.69,
median=1.0, p75=1.0, p95=50.0, p98=1000.0, p99=5000.0, p999=10000.0

type=TIMER, name=transactionTimer, count=107,
min=0.557524, max=1272.75, mean=27.16, stddev=152.57,
median=0.85, p75=1.24, p95=41.25, p98=283.50, p99=856.19, p999=1272.75,
mean_rate=36.32, rate_unit=events/second, duration_unit=milliseconds
```

For a high-performance data access layer, statistics and metrics becomes mandatory requirements. The Hibernate statistics mechanism is a very powerful tool, allowing the development team to get a better insight into Hibernate inner workings.

8.4 Statement logging

An ORM tool can automatically generate DML statements, and it is the application developer responsibility to validate both their effectiveness as well as their overall performance impact. Deferring the SQL statement validation until the data access layer starts showing performance issues is risky, and it can even impact development cost. For this reason, SQL statement logging becomes relevant from the early stages of application development.

When a business logic is implemented, the Definition of Done should include a review of all the associated data access layer operations. Following this rule can save much hassle when the enterprise system is deployed into production.

Although the JPA 2.1 does not feature a standard configuration property for logging SQL statements, most JPA implementations support this feature through framework-specific setups. For this purpose, Hibernate defines the following configuration properties:

Table 8.4: Connection release modes

Property	Description
`hibernate.show_sql`	Prints SQL statements to the console
`hibernate.format_sql`	Formats SQL statements before being logged or printed to the console
`hibernate.use_sql_comments`	Adds comments to the automatically generated SQL statement

Using the `System` console for logging is bad practice, a logging framework (e.g. Logback or Log4j) being a better alternative for it supports configurable appenders and logging levels.

Because it prints to the console, the `hibernate.show_sql` property should be avoided.

Hibernate logs all SQL statements on a `debug` level in the `org.hibernate.SQL` logging hierarchy.

To enable statement logging, the following Logback logger must be added to the associated configuration file:

```
<logger name="org.hibernate.SQL" level="debug"/>
```

Because Hibernate uses `PreparedStatement(s)` exclusively, the bind parameter values are not available when the statement gets printed into the log:

```
INSERT INTO post (title, version, id) VALUES (?, ?, ?)
```

Although bind parameters might be logged separately (e.g. `org.hibernate.type.descriptor.sql`), the most straight-forward way of logging SQL statements along with their runtime bind parameter values is to use an external `DataSource` proxy. Because the proxy intercepts all statement executions, the bind parameter values can be introspected and printed as well.

8.4.1 Statement formatting

By default, every SQL statement, no matter how long, is written as a single line of text. To increase readability, Hibernate can transform SQL statements in a human-readable format that spans over multiple log lines. This feature can be activated by setting the following configuration property:

```
<property name="hibernate.format_sql" value="true" />
```

With this setting in place, the previous statement can be formatted as follows:

```
insert
    into
        post
        (title, version, id)
    values
        (?, ?, ?)
```

Although formatting statements can improve readability, this setting is only suitable for the development phase. In a production system, logs are often parsed and aggregated in a centralized system, and the multi-line statement format can impact the log parsing mechanism. Once aggregated, logged queries can be formatted prior to being displayed in the application performance monitoring user interface.

The `hibernate.format_sql` property applies to logged statements only, and it does not propagate to the underlying JDBC `Driver` (SQL statements are still sent as single lines of text).

This way, the statement formatting does not have any effect when statements are logged through an external `DataSource` proxy.

8.4.2 Statement-level comments

Besides formatting, Hibernate can explain the statement generation process by appending SQL-level comments into the statement body. This feature allows the application developer to get a better understanding of the following processes:

- the entity state transition that triggered the currently executing statement
- the reason for choosing a join when fetching a given result set
- the explicit locking mechanism employed by the current statement.

By default, Hibernate does not append any SQL comment in the automatically generated statements, and, to enable this mechanism, the following Hibernate property must be configured:

```
<property name="hibernate.use_sql_comments" value="true" />
```

When persisting a `Post` entity, Hibernate explains the entity state transition associated with this particular statement through the following comment:

```
/* insert com.vladmihalcea.book.hpjp.util.providers.BlogEntityProvider$Post */
INSERT INTO post (title, version, id) VALUES (?, ?, ?)
```

As opposed to SQL statement formatting, SQL comments are generated not only during logging as they propagate to the underlying `Driver` as well.

Although it might be a useful technique for debugging purposes, in a production environment, it is better to leave it disabled, to reduce the database request networking overhead.

8.4.3 Logging parameters

Either the JDBC `Driver` or the `DataSource` must be proxied to intercept statement executions and log them along with the actual parameter values. Besides statement logging, a JDBC proxy can provide other cross-cutting features like long-running query detection or custom statement execution listeners.

8.4.3.1 DataSource-proxy

A lesser-known JDBC logging framework, datasource-proxy[9] provides support for custom JDBC statement execution listeners. In Java EE, not all application servers allow configuring an external `DataSource`, as they rely on their own custom implementations that bind the user supplied JDBC `Driver`. Because it can only decorate a `DataSource`, datasource-proxy might not be suitable for all Java EE environments.

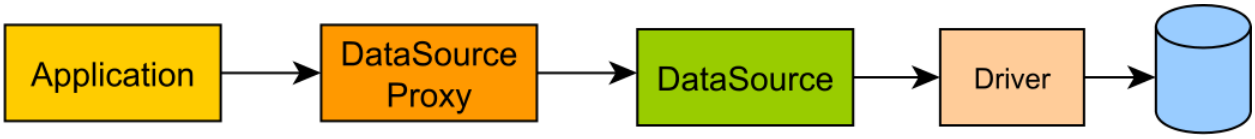

Figure 8.4: DataSource-Proxy Architecture

On the other hand, the programmatic configuration support fits the Java-based configuration approach taken by most modern Spring applications:

```
@Bean
public DataSource dataSource() {
    SLF4JQueryLoggingListener loggingListener = new SLF4JQueryLoggingListener();
    loggingListener.setQueryLogEntryCreator(new InlineQueryLogEntryCreator());
    return ProxyDataSourceBuilder
        .create(actualDataSource())
        .name(DATA_SOURCE_PROXY_NAME)
        .listener(loggingListener)
        .build();
}
```

[9] https://github.com/ttddyy/datasource-proxy

In the following example, datasource-proxy is used to log a batch insert of three `PreparedStatement(s)`. Although normally a batch is printed in a single line of log, the output was split into multiple lines to fit the current page layout.

```
Name:DATA_SOURCE_PROXY, Time:6, Success:True,
Type:Prepared, Batch:True, QuerySize:1, BatchSize:3,
Query:["insert into post (title, version, id) values (?, ?, ?)"],
Params:[(Post no. 0, 0, 0), (Post no. 1, 0, 1), (Post no. 2, 0, 2)]
```

Not only the bind parameter values are now present, but, because they are grouped all together, it is very easy to visualize the batching mechanism too.

With the custom statement listener support, datasource-proxy allows building a query count validator to assert the auto-generated statement count and, therefore, prevent N+1 query problems during the development phase.

8.4.3.2 P6Spy

P6Spy[10] was released in 2002, in an era when J2EE application servers were ruling the world of enterprise systems. Because Java EE application servers do not allow programmatic `DataSource` configuration, P6Spy supports a declarative configuration approach (through a `spy.properties` file).

P6Spy offers support for proxying both a JDBC `Driver` (which is suitable for Java EE applications) or a JDBC `DataSource` (supported by some Java EE containers and common practice for Spring enterprise applications).

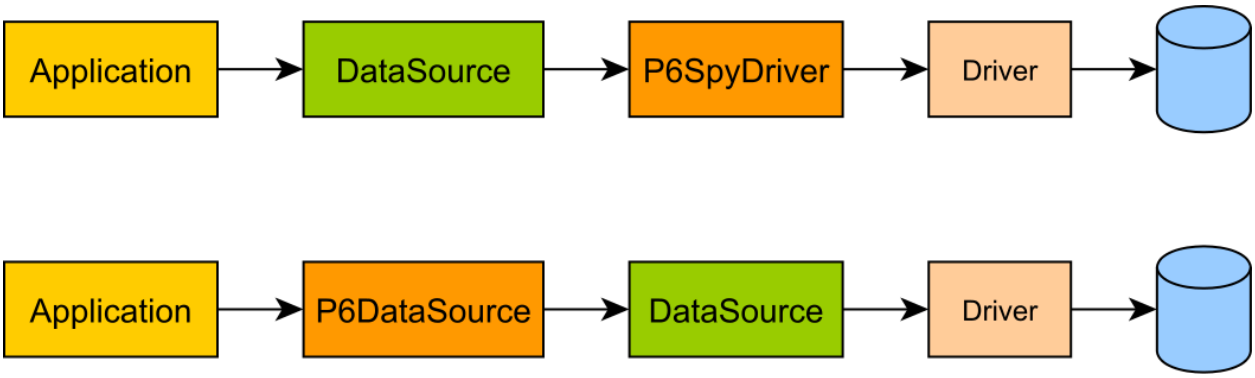

Figure 8.5: P6Spy Architecture

[10] https://github.com/p6spy/p6spy

Running the previous example gives the following output (formatting was also applied):

```
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 0', 0, 0)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 1', 0, 1)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
p6spy - 1448122491812|5|statement|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
```

In the order of their occurrence, the output is built out of the following columns:

Table 8.5: P6Spy output

Field	Description
Timestamp	The statement execution timestamp
Execution time	The statement execution duration (in milliseconds)
Category	The current statement category (e.g. statement, batch)
Connection	The database connection identifier (as assigned by P6Spy)
Original statement	The original statement that was intercepted by P6Spy
Formatted statement	The statement with all parameter placeholders replaced with the actual bind values

The first three lines are associated with adding statements to the batch, while the fourth line is logging the actual batch execution (which also explains the *execution time* column value).

One very useful configuration is the `outagedetection` property, which can detect long-running statements.

9. Mapping Types and Identifiers

JPA addresses the Object/Relational mismatch by associating Java object types to database structures Assuming there is a `task` database table having four columns (e.g. `id`,`created_by`, `created_on`, and `status`), the JPA provider must map it to Domain Model consisting of two Java class (e.g. `Task` and `Change`).

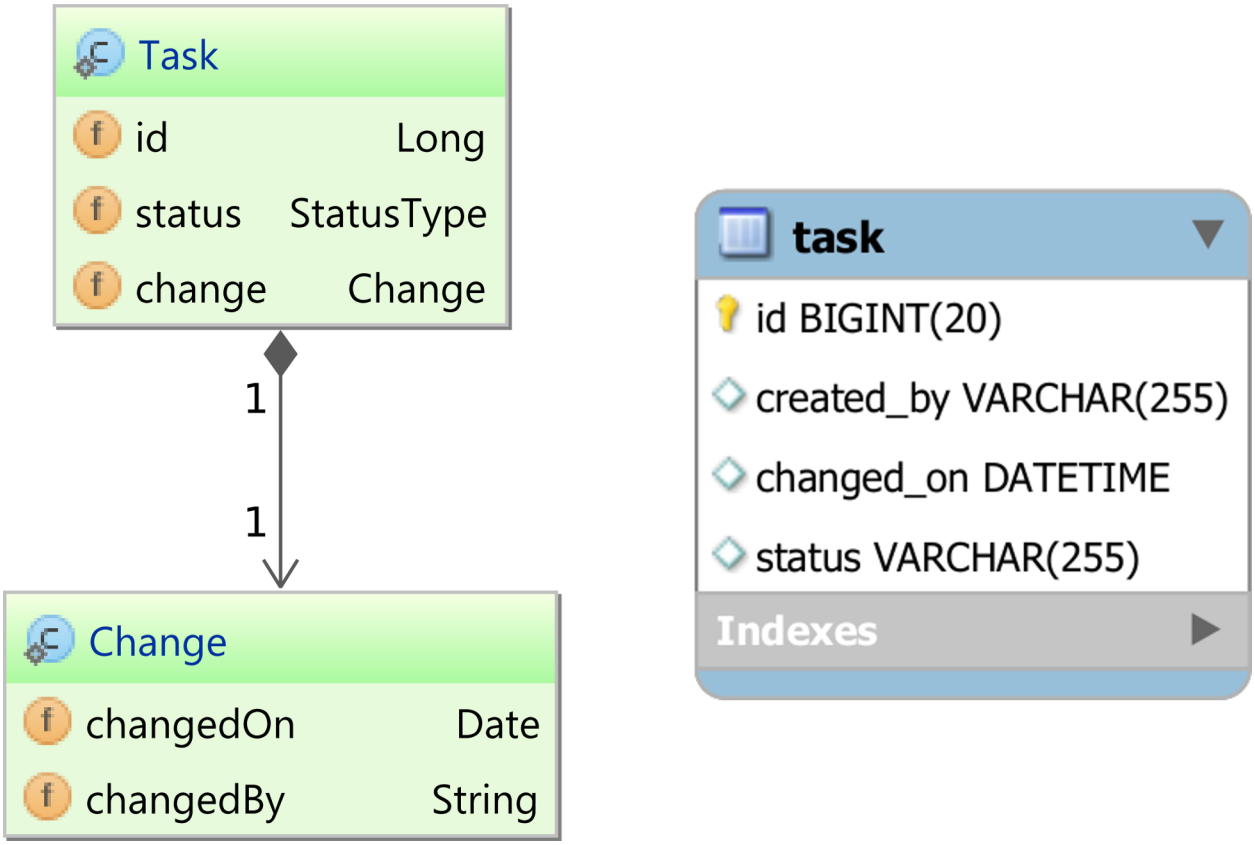

Figure 9.1: Type Mapping

JPA uses three main Object-Relational mapping elements: *type*, *embeddable*, and *entity*. In the previous diagram, the `Task` object is an entity, while the `Change` object is an embeddable type.

Both the entity and the embeddable group multiple Domain Model attributes by relying on Hibernate `Type(s)` to associate database column types with Java value objects (e.g. `String`, `Integer`, `Date`). The major difference between an entity and an embeddable is the presence of an identifier, which is used to associate a database table unique key (usually the primary key) with a Domain Model object attribute.

Although it is common practice to map all database columns, this is not a strict requirement. Sometimes it is more practical to use a root entity and several sub-entities, so each business case fetches just as much info as needed (while still benefiting from entity state management).

Identifiers are mandatory for entity elements, and an embeddable type is forbidden to have an identity of its own. Knowing the database table and the column that uniquely identifies any given row, Hibernate can correlate database rows with Domain Model entities.

An embeddable type groups multiple attributes in a single reusable component.

```
@Embeddable
public class Change {
    @Column(name = "changed_on")
    private Date changedOn;

    @Column(name = "created_by")
    private String changedBy;
}
```

The Domain Model can share state between multiple entities either by using inheritance or composition. Embeddable types can reuse state through composition.

The composition association[1], defined by UML, is the perfect analogy for the relationship between an entity and an embeddable. When an entity includes an embeddable type, all its attributes become part of the owner entity.

The embeddable object cannot define its own identifier because otherwise, the entity has more than one identities. Lacking an identifier, the embeddable object cannot be managed by a Persistence Context, and its state is controlled by its parent entity.

Because they can have a significant impact on the overall application performance, the rest of the chapter discusses types and identifiers in greater detail.

[1] https://en.wikipedia.org/wiki/Object_composition

9.1 Types

For every supported database type, JDBC defines a `java.sql.JDBCType` enumeration. Since it builds on top of JDBC, Hibernate does the mapping between JDBC types and their associated Java counterparts (primitives or Objects).

9.1.1 Primitive types

Table 9.1: Primitive Types

Hibernate type	JDBC type	Java type
`BooleanType`	`BIT`	`boolean`, `Boolean`
`NumericBooleanType`	`INTEGER` (e.g. `0`, `1`)	`boolean`, `Boolean`
`TrueFalseType`	`CHAR` (e.g. `'F'`, `'f'`, `'T'`, `'t'`)	`boolean`, `Boolean`
`YesNoType`	`CHAR` (e.g. `'N'`, `'n'`, `'Y'`, `'y'`)	`boolean`, `Boolean`
`ByteType`	`TINYINT`	`byte`, `Byte`
`ShortType`	`SMALLINT`	`short`, `Short`
`CharacterType`	`CHAR`	`char`, `Character`
`CharacterNCharType`	`NCHAR`	`char`, `Character`
`IntegerType`	`INTEGER`	`int`, `Integer`
`LongType`	`BIGINT`	`long`, `Long`
`FloatType`	`FLOAT`	`float`, `Float`
`DoubleType`	`DOUBLE`	`double`, `Double`
`CharArrayType`	`VARCHAR`	`char[]`, `Character[]`

From one database system to another, the boolean type can be represented either as a `BIT`, `BYTE`, `BOOLEAN` or `CHAR` database type, so defines four `Type(s)` to resolve the boolean primitive type.

Only non-nullable database columns can be mapped to Java primitives (`boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, `double`). For mapping nullable columns, it is better to use the primitive wrappers instead (`Boolean`, `Byte`, `Short`, `Char`, `Integer`, `Long`, `Float`, `Double`).

9.1.2 String types

A Java `String` can consume as much memory as the Java Heap has available. On the other hand, database systems define both limited-size types (`VARCHAR` and `NVARCHAR`) and unlimited ones (`TEXT`, `NTEXT`, `BLOB`, and `NCLOB`).

To accommodate this mapping discrepancy, Hibernate defines the following `Type(s)`:

Table 9.2: String Types

Hibernate type	JDBC type	Java type
`StringType`	`VARCHAR`	`String`
`StringNVarcharType`	`NVARCHAR`	`String`
`TextType`	`LONGVARCHAR`	`String`
`NTextType`	`LONGNVARCHAR`	`String`
`MaterializedClobType`	`CLOB`	`String`
`MaterializedNClobType`	`NCLOB`	`String`

9.1.3 Date and Time types

When it comes to time, there are multiple Java or database representations, which explains the vast number of time-related Hibernate `Type(s)`.

Table 9.3: Date and Time Types

Hibernate type	JDBC type	Java type
`DateType`	`DATE`	`Date`
`TimeType`	`TIME`	`Time`
`TimestampType`	`TIMESTAMP`	`Timestamp`, `Date`
`DbTimestampType`	`TIMESTAMP`	`Timestamp`, `Date`
`CalendarType`	`TIMESTAMP`	`Calendar`, `GregorianCalendar`
`CalendarDateType`	`DATE`	`Calendar`, `GregorianCalendar`
`CalendarTimeType`	`TIME`	`Calendar`, `GregorianCalendar`
`TimeZoneType`	`VARCHAR`	`TimeZone`

Handling time is tricky because of various time zones, leap seconds and daylight saving conventions. Storing timestamps in UTC (Coordinated Universal Time) and doing time zone transformations in the data layer is common practice.

9.1.4 Numeric types

Oracle can represent numbers up to 38 digits, therefore only fitting in a `BigInteger` or a `BigDecimal` (`java.lang.Long` and `java.lang.Double` can only store up to 8 bytes).

Table 9.4: Numeric Types

Hibernate type	JDBC type	Java type
`BigIntegerType`	`NUMERIC`	`BigInteger`
`BigDecimalType`	`NUMERIC`	`BigDecimal`

9.1.5 Binary types

For binary types, most database systems offer multiple storage choices (e.g. `RAW`, `VARBINARY`, `BYTEA`, `BLOB`, `CLOB`). In Java, the data access layer can use an array of `byte(s)`, a JDBC `Blob` or `Clob`, or even a `Serializable` type, if the Java object was marshaled prior to being saved to the database.

Table 9.5: Binary Types

Hibernate type	JDBC type	Java type
`BinaryType`	`VARBINARY`	`byte[],Byte[]`
`BlobType`	`BLOB`	`Blob`
`ClobType`	`CLOB`	`Clob`
`NClobType`	`NCLOB`	`Clob`
`MaterializedBlobType`	`BLOB`	`byte[],Byte[]`
`ImageType`	`LONGVARBINARY`	`byte[],Byte[]`
`SerializableType`	`VARBINARY`	`Serializable`
`SerializableToBlobType`	`BLOB`	`Serializable`

9.1.6 UUID types

There are various ways of persisting a Java `UUID` (Universally Unique Identifier), and, based on the memory footprint, the most efficient storage types are the database-specific UUID column types.

Table 9.6: UUID Types

Hibernate type	JDBC type	Java type
`UUIDBinaryType`	`BINARY`	`UUID`
`UUIDCharType`	`VARCHAR`	`UUID`
`PostgresUUIDType`	`OTHER`	`UUID`

When not natively supported, a `BINARY` type requires fewer bytes than a `VARCHAR`, so the associated index has a smaller memory footprint too.

9.1.7 Other types

Hibernate can also map Java `Enum(s)`, `Class`, `URL`, `Locale` and `Currency` too.

Table 9.7: Other Types

Hibernate type	JDBC type	Java type
`EnumType`	`CHAR`, `LONGVARCHAR`, `VARCHAR`	`Enum`
	`INTEGER`, `NUMERIC`, `SMALLINT`, `TINYINT`, `BIGINT`, `DECIMAL`, `DOUBLE`, `FLOAT`	
`ClassType`	`VARCHAR`	`Class`
`CurrencyType`	`VARCHAR`	`Currency`
`LocaleType`	`VARCHAR`	`Locale`
`UrlType`	`VARCHAR`	`URL`

9.1.8 Custom types

Not only that it has a very rich set of data types, but PostgreSQL allows adding custom types as well (using the `CREATE DOMAIN`[2] DDL statement). Choosing the appropriate database type for each Domain Model field can really make a difference in terms of data access performance. Although there is a great variety of built-in `Type(s)`, the application developer is not limited to the off-the-shelf ones only, and new `Type(s)` can be added without too much effort.

In the following example, the business logic requires monitoring access to an enterprise application. For this purpose, the data access layer stores the IP (Internet Protocol) addresses of each logged-in user.

Assuming this internal application uses the IPv4 protocol only, the IP addresses are stored in the Classless Inter-Domain Routing format (e.g. `192.168.123.231/24`). PostgreSQL can store IPv4 addresses either in a `cidr` or `inet` type, or it can use a `VARCHAR(18)` column type.

The `VARCHAR(18)` column requires 18 characters, and, assuming a UTF-8 encoding, each IPv4 address needs at most 18 bytes. The smallest size address (e.g. `0.0.0.0/0`) taking 9 characters, the `VARCHAR(18)` approach requires between 9 and 18 characters for each IPv4 address.

The `inet` type is specially designed for IPv4 and IPv6 network addresses, and it also supports various network address specific operators (e.g. `<`, `>`, `&&`), as well as other address transforming

[2] http://www.postgresql.org/docs/9.5/static/sql-createdomain.html

functions (e.g. `host(inet)`, `netmask(inet)`). As opposed to the `VARCHAR(18)` approach, the `inet` type requires only 7 bytes for each IPv4 address.

For it has a more compact size (the index can better fit into memory) and supporting many specific operators, the `inet` type is a much more attractive choice. Although, by default, Hibernate does not support `inet` types, adding a custom Hibernate `Type` is a straightforward task. The IPv4 address is encapsulated in its own wrapper, which can also define various address manipulation functions too.

```
public class IPv4 implements Serializable {

    private final String address;

    public IPv4(String address) {
        this.address = address;
    }

    public String getAddress() {
        return address;
    }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return Objects.equals(address, IPv4.class.cast(o).address);
    }

    @Override public int hashCode() {
        return Objects.hash(address);
    }

    public InetAddress toInetAddress() throws UnknownHostException {
        return Inet4Address.getByName(address);
    }
}
```

When an entity wants to change an `IPv4` field, it must provide a new object instance. An immutable type is much easier to handle since its internal state does not change throughout the currently running Persistence Context.

All custom types must implement the `UserType` interface, and, since the `ImmutableType` takes care of most `UserType` implementation details, the `IPv4Type` can focus on type-specific conversation logic.

```
public class IPv4Type extends ImmutableType<IPv4> {

    public IPv4Type() {
        super(IPv4.class);
    }

    @Override public int[] sqlTypes() { return new int[]{ Types.OTHER}; }

    @Override public IPv4 get(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException {
        String ip = rs.getString(names[0]);
        return (ip != null) ? new IPv4(ip) : null;
    }

    @Override public void set(PreparedStatement st, IPv4 value, int index,
        SessionImplementor session) throws SQLException {
        if (value == null) {
            st.setNull(index, Types.OTHER);
        } else {
            PGobject holder = new PGobject();
            holder.setType("inet");
            holder.setValue(value.getAddress());
            st.setObject(index, holder);
        }
    }
}
```

The `get()` method is used to map the `inet` field to an `IPv4` object instance, while the `set()` is used for transforming the `IPv4` object to the PostgreSQL JDBC driver `inet` equivalent.

`Types.OTHER` is used for mapping database types not supported by JDBC.

```java
public abstract class ImmutableType<T> implements UserType {
    private final Class<T> clazz;

    protected ImmutableType(Class<T> clazz) { this.clazz = clazz; }

    @Override public Object nullSafeGet(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException {
        return get(rs, names, session, owner);
    }

    @Override public void nullSafeSet(PreparedStatement st, Object value,
        int index, SessionImplementor session) throws SQLException {
        set(st, clazz.cast(value), index, session);
    }

    @Override public Class<T> returnedClass() { return clazz; }

    @Override public boolean equals(Object x, Object y) {
        return Objects.equals(x, y);
    }

    @Override public int hashCode(Object x) { return x.hashCode(); }

    @Override public Object deepCopy(Object o) { return o; }

    @Override public boolean isMutable() { return false; }

    @Override public Serializable disassemble(Object o) {
        return (Serializable) o;
    }

    @Override public Object assemble(Serializable o, Object owner) { return o; }

    @Override
    public Object replace(Object o, Object target, Object owner) { return o; }

    protected abstract T get(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException;

    protected abstract void set(PreparedStatement st, T value, int index,
        SessionImplementor session) throws SQLException;
}
```

The `@Type` annotation instructs Hibernates to use the `IPv4Type` for mapping the `IPv4` field.

```
@Entity
public class Event {

    @Id @GeneratedValue
    private Long id;

    @Type(type = "com.vladmihalcea.book.hpjp.hibernate.type.IPv4Type")
    @Column(name = "ip", columnDefinition = "inet")
    private IPv4 ip;

    public Event() {}

    public Event(String address) {
        this.ip = new IPv4(address);
    }

    public Long getId() {
        return id;
    }

    public IPv4 getIp() {
        return ip;
    }

    public void setIp(String address) {
        this.ip = new IPv4(address);
    }
}
```

GiST operators

PostgreSQL 9.4 added GiST operator support[a] for `inet` and `cidr` column types. To enable this feature, a GiST index with the `inet_ops` operator class must be created on the associated `inet` columns.

```
CREATE INDEX ON event USING gist (ip inet_ops)
```

[a] http://www.postgresql.org/docs/current/static/gist-builtin-opclasses.html

Managing `Event(s)` is easy when Hibernate takes care of the underlying type conversation.

```
final AtomicReference<Event> eventHolder = new AtomicReference<>();

doInJPA(entityManager -> {
    entityManager.persist(new Event());
    Event event = new Event("192.168.0.231");
    entityManager.persist(event);
    eventHolder.set(event);
});

doInJPA(entityManager -> {
    Event event = entityManager.find(Event.class, eventHolder.get().getId());
    event.setIp("192.168.0.123");
});
```

Running the previous example generates the following SQL statements:

```
INSERT INTO event (ip, id) VALUES (NULL(OTHER), 1)
INSERT INTO event (ip, id) VALUES (`192.168.0.231`, 2)

SELECT e0_.id as id1_0_0_, e0_.ip as ip2_0_0_
FROM event e0_
WHERE e0_.id = 2

UPDATE event SET ip=`192.168.0.123` WHERE id = 2
```

One of the best aspects of using database-specific types is getting access to advanced querying capabilities. Because the GiST index allows `inet_ops` operators, the following query can be used to check if an `Event` was generated for a given subnetwork:

```
Event matchingEvent = (Event) entityManager.createNativeQuery(
    "SELECT e.* " +
    "FROM event e " +
    "WHERE " +
    "   e.ip && CAST(:network AS inet) = TRUE", Event.class)
.setParameter("network", "192.168.0.1/24")
.getSingleResult();

assertEquals("192.168.0.123", matchingEvent.getIp().getAddress());
```

9.2 Identifiers

All database tables must have a primary key column, so each row can be uniquely identified (the primary key must be both `UNIQUE` and `NOT NULL`).

> Although the SQL standard does not impose primary keys to be immutable, it is more practical[a] to avoid changing them.
>
> [a] https://asktom.oracle.com/pls/asktom/f?p=100:11:0::::P11_QUESTION_ID:5773459616034

The primary key can have a meaning in the real world, in which case it is a *natural* key, or it can be generated synthetically, in which case it is called a *surrogate* identifier.

For natural keys, unicity is enforced by a real-world unique sequence generator (e.g. National Identification Numbers, Social Security Numbers, Vehicle Identification Numbers). In reality, natural unique numbers might pose problems when the unique constraints do not hold true anymore. For example, a National Identification Number might yield unique numbers, but if the enterprise system must accommodate users coming from multiple countries, it is possible that two different countries assigned the same identifier.

The natural key can be composed of one or multiple columns. Compound natural keys might incur an additional performance penalty because multi-column joins are slower than single-column ones, and multi-column indexes have a bigger memory footprint too.

Natural keys must be sufficiently long to accommodate as many identifiers as the system needs throughout its lifecycle. Because primary keys are often indexed, the longer the key, the more memory an index entry requires. Each joined table includes a foreign key mirroring the parent primary key, and foreign keys are frequently indexed as well.

> **Index memory impact**
>
> Fixed-size non-numerical keys (e.g. `CHAR`, `VARCHAR`) are less efficient than numerical ones (e.g. `INTEGER`, `BIGINT`) both for joining (a simple key performs better than a compound one) or indexing (the more compact the data type, the less memory space is required by an associated index).
>
> A `CHAR(17)` natural key (e.g. Vehicle Identification Number) requires 17 characters (17 bytes when using ASCII characters and a UTF-8 encoding) as opposed to 4 bytes (32 bit `INTEGER`) or 8 bytes (64 bit `BIGINT`).